

01 — HANA DB Connection (Deep Dive)

Module purpose: every other template depends on this one — a live connection object to SAP HANA Cloud. This sheet covers all four connection-related files: two ways to authenticate (.env locally vs. BTP Cloud Foundry bound service), a second driver style for ML workloads, and a reusable health check.

File 1 — connect_env.py (local development)

```
import os
from dotenv import load_dotenv      # reads KEY=VALUE lines from a .env file into os.environ
from hdbcli import dbapi           # SAP's official low-level HANA driver (DB-API 2.0 style)

load_dotenv()                      # call once, near the top of the module

def connect() -> dbapi.Connection:
    required = ["HANA_HOST", "HANA_PORT", "HANA_USER", "HANA_PASSWORD"]
    missing = [v for v in required if not os.getenv(v)]
    if missing:                     # fail fast with a clear error, not a cryptic
        ↪ driver exception
        raise EnvironmentError(f"Missing in .env: {missing}")

    conn = dbapi.connect(
        address = os.getenv("HANA_HOST"),          # e.g. xxx.hanacloud.ondemand.com
        port    = int(os.getenv("HANA_PORT", 443)), # 443 = HANA Cloud's default SQL port
        user    = os.getenv("HANA_USER"),
        password = os.getenv("HANA_PASSWORD"),
        encrypt  = "true",                          # TLS — mandatory on HANA Cloud,
        ↪ connection is refused without it
        sslTrustStore = os.getenv("HANA_CERTIFICATE", ""), # optional custom CA cert (blank
        ↪ on BTP trial)
    )
    return conn                                # caller owns lifecycle — must call
    ↪ conn.close()
```

Working explanation: load_dotenv() populates the process environment from a local .env file so the same os.getenv() calls work identically whether the value came from a file (local dev) or was injected by the platform (CI/CD, container env). The missing check exists so a misconfigured .env fails with one readable message instead of a raw driver stack trace several calls later. dbapi.connect() opens a TCP+TLS session and authenticates in one call — nothing is lazy here, so connect() either returns a working connection or raises immediately.

File 2 — connect_btp_cf.py (Cloud Foundry deployment)

```
from cfenv import AppEnv           # reads Cloud Foundry's VCAP_SERVICES/VCAP_APPLICATION
↪ env vars
from hdbcli import dbapi

CF_SERVICE_LABEL = "hana"          # must match the label shown by `cf services`

def connect() -> dbapi.Connection:
    cf_env = AppEnv()
    svc = cf_env.get_service(label=CF_SERVICE_LABEL) # finds the bound service instance by
    ↪ label
```

```

if svc is None:
    raise EnvironmentError(f"HANA service '{CF_SERVICE_LABEL}' not found in
        ↪ VCAP_SERVICES.")

creds = svc.credentials                # dict auto-parsed from the bound service's JSON
↪ credentials
conn = dbapi.connect(
    address=creds["host"], port=int(creds.get("port", 443)),
    user=creds["user"], password=creds["password"],
    encrypt="true", sslTrustStore=creds.get("certificate", ""),
)
return conn

```

Working explanation: when an app is cf pushed and cf bind-serviced to a HANA instance, Cloud Foundry injects a VCAP_SERVICES environment variable containing a JSON blob of credentials — cfenv parses that blob so you never read raw env vars by hand. This is the **production** counterpart to connect_env.py; same dbapi.connect() call at the bottom, only where the credentials come from differs. This is a very common interview pattern: “local secret via .env, production secret via a bound platform service.”

File 3 — connect_hana_ml.py (ConnectionContext for ML / PAL)

```

import hana_ml.dataframe as dataframe

def connect() -> dataframe.ConnectionContext:
    cc = dataframe.ConnectionContext(
        address=os.getenv("HANA_HOST"), port=int(os.getenv("HANA_PORT", 443)),
        user=os.getenv("HANA_USER"), password=os.getenv("HANA_PASSWORD"),
        encrypt="true", sslValidateCertificate="false",
    )
    return cc

# Usage differs from hdbcli — hana-ml gives you a DataFrame API, not raw cursors:
result = cc.sql("SELECT CURRENT_UTCTIMESTAMP FROM DUMMY").collect() # .collect() pulls
↪ results into pandas

```

Working explanation: hana_ml is a **different library** from hdbcli — it wraps the same underlying connection but exposes a pandas-like DataFrame API plus access to HANA’s PAL (Predictive Analysis Library) machine-learning algorithms and the HANA-native vector/ML functions. You’d reach for hana_ml.ConnectionContext when doing dataframe-style analytics or calling PAL algorithms; you’d reach for plain hdbcli (files 1-2) for straightforward SQL/cursor work.

File 4 — health_check.py (works with either connection style)

```

def health_check(conn) -> dict:
    result = {"status": "FAILED", "timestamp": None, "version": None, "error": None}
    cursor = None
    try:
        cursor = conn.cursor()
        cursor.execute("SELECT CURRENT_UTCTIMESTAMP FROM DUMMY") # DUMMY = HANA's
↪ built-in 1-row table
        result["timestamp"] = str(cursor.fetchone()[0])

```

```

        cursor.execute("SELECT VALUE FROM M_SYSTEM_OVERVIEW WHERE NAME = 'Version'") # HANA
↪ server version
        row = cursor.fetchone()
        result["version"] = str(row[0]) if row else "Unknown"
        result["status"] = "OK"
    except Exception as e:
        result["error"] = str(e) # never let a health check itself crash
↪ the caller
    finally:
        if cursor: cursor.close()
    return result

```

Working explanation: two lightweight system queries prove the DB is reachable AND readable — DUMMY for “is it alive,” M_SYSTEM_OVERVIEW for version/build info useful in support tickets. Everything is wrapped in try/except/finally so a health check function can never itself throw — it always returns a structured dict the caller can log or expose on a /health endpoint.

Key Points

- encrypt="true" is **mandatory** on HANA Cloud — there is no unencrypted connection option.
- Two credential-sourcing strategies, same dbapi.connect() call: .env (local) vs. VCAP_SERVICES via cfenv (Cloud Foundry) — swapping between them is a one-line import change everywhere else in the codebase (from connect_env import connect → from connect_btp_cf import connect).
- hdbcli (raw SQL/cursors) vs. hana_ml (DataFrame/PAL/ML) are two different client libraries for two different jobs — know when to use each.
- DUMMY is HANA’s equivalent of SELECT 1 — a guaranteed single-row table for pings.
- Always close both the **cursor** (finally: cursor.close()) and the **connection** (conn.close() when the app shuts down) — connections are a limited, billable resource.
- One connection object is created once per app/session and **reused** by every downstream template (query agent, vector store, RAG, chatbot) — never reconnect per request.

Interview Q&A

Q: Why does the health check wrap everything in try/except instead of letting exceptions propagate? A: A health check’s job is to *report* status, not to crash — callers (monitoring dashboards, /health HTTP endpoints) expect a structured result even when the DB is down, not an unhandled exception.

Q: What’s the actual difference between hdbcli and hana_ml, concretely? A: hdbcli is DB-API 2.0 — you write raw SQL strings and get back cursor rows. hana_ml wraps that with a pandas-DataFrame-like interface (ConnectionContext.sql(...).collect()) plus bindings to HANA’s server-side PAL machine learning algorithms — useful when doing analytics/ML, overkill for simple CRUD.

Q: How would you structure credential management across local dev, CI, and production? A: Local: .env + python-dotenv (never committed). CI: injected secrets as env vars. Production on Cloud Foundry: a bound service resolved via VCAP_SERVICES/cfenv. Same connect() interface, different credential source — the rest of the app never needs to know which one is active.

Q: Why is sslTrustStore sometimes blank? A: On BTP trial/free-tier HANA Cloud instances, the platform’s default CA is already trusted by the client OS trust store, so no custom certificate path is required; production or on-prem-linked setups may require pointing at a specific CA certificate file.